



T.C. İSTANBUL KÜLTÜR ÜNİVERSİTESİ

COM5005 – WEB PROGRAMMING

LAB 13 – AJAX and JQUERY

After completing this Lab, you will be able to

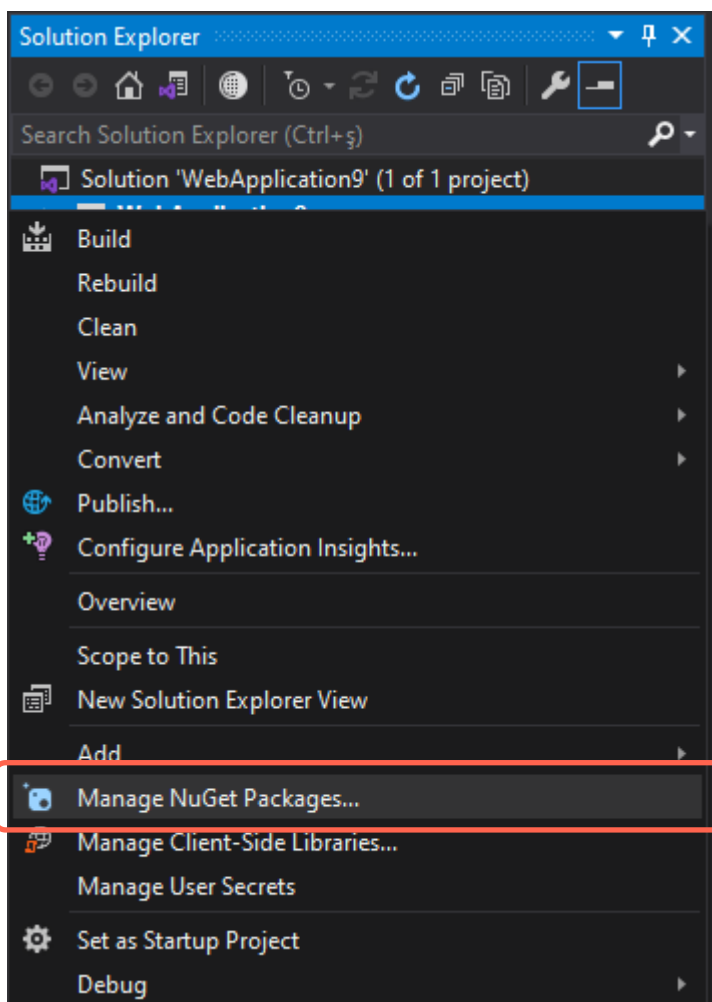
- Installing JQuery-Unobstrusive-AJAX
- Create “Strongly Typed Ajax” (Asynchronous)
- Pure Html Forms with Ajax and JQuery
- Search Page using Ajax and JQuery

PROCEDURE 1 – Installing JQuery-Unobstrusive-AJAX

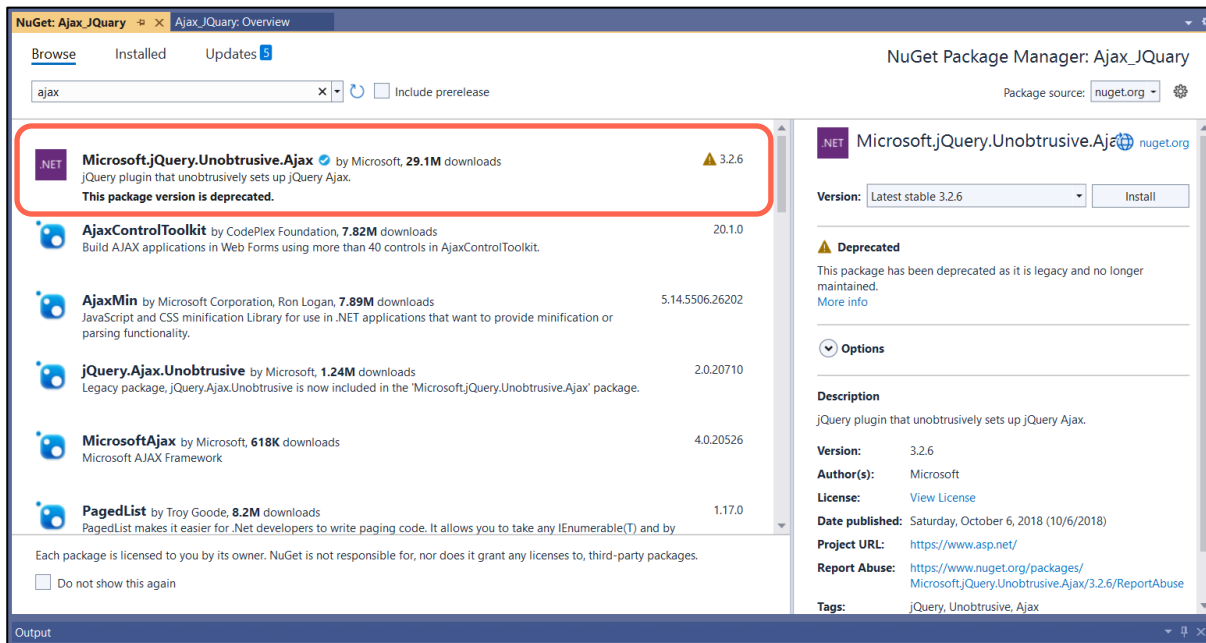
Asynchronous AJAX form is a very magical way to submit data to the controller without happening page load. Asynchronous AJAX Forms simply post back the data to the controllers and update the only that part of the page, which has to display output.

To make this happen, we will use JQuery-Unobstrusive-AJAX. This is a great feature which is launched in MVC 3. It helps you to create AJAX Form without writing bunch of javascript code. Before creating Asynchronous AJAX Form you need to add JQuery-Unobstrusive-AJAX in your project. Adding is very easy, and just follows the steps.

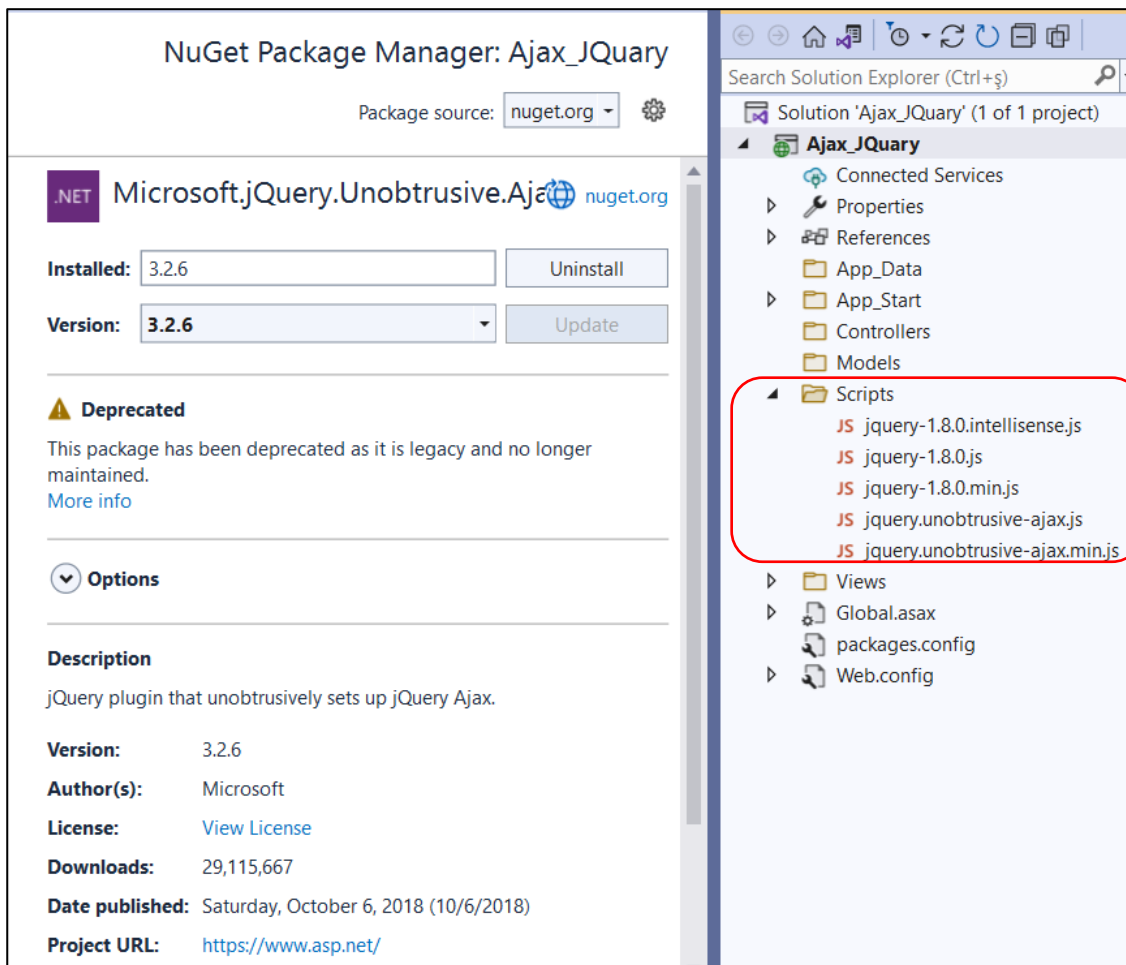
Step 1 – Right-click on your Project name in Solution Explorer and click **Manage Nuget Package**.



Step 2 – Go to Browse and search for ajax. Find and Install Microsoft-JQuery-Unobtrusive-Ajax.



Step 3 – After installing this you can see it in Script folder.



Now, your project is ready to use **JavaScript** and **AJAX**.

PROCEDURE 2 – Create “Strongly Typed Ajax” (Asynchronous)

Step 1 – Let’s create new project Forms and Controller. Right click on the Models folder and add a new **StudentModel.cs** class.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;

namespace Ajax_JQuery.Models
{
    0 references
    public class StudentModel
    {
        0 references
        public int Id { get; set; }
        0 references
        public string Name { get; set; }
        0 references
        public bool Addon { get; set; }
    }
}
```

Step 2 – Create Home Controller. Let's create **Index ActionResult** as get method in Home Controller. This method should return Index.html view as default.

Next, let's create the **Form ActionResult** method. This method should create object from StudentModel class and use class items.

```
// GET: Home
public ActionResult Index()
{
    return View();
}

[HttpPost]
public ActionResult Form(models.StudentModel sm)
{
    if (ModelState.IsValid)
    {
        System.Text.StringBuilder sb = new System.Text.StringBuilder();
        sb.Append("ID: " + sm.Id + "<br />");
        sb.Append("Name: " + sm.Name + "<br />");
        sb.Append("Addon: " + sm.Addon + "<br />");
        return Content(sb.ToString());
    }
    else
    {
        return View("Index");
    }
}
```

Step 3 – Create Form in **Index.cshtml**. Right click on the Home file under the View folder and add a new **Index.cshtml** view.

```
@model Ajax_JQuery_0.Models.StudentModel
<script src="@Url.Content("~/Scripts/jquery-1.8.0.min.js")"
type="text/javascript"></script>
<script src="@Url.Content("~/Scripts/jquery.unobtrusive-ajax.js")"
type="text/javascript"></script>

<h4 id="id1" style="color:purple"></h4>
<hr />
<h3><b>Forms - Strongly Typed AJAX (Asynchronous)</b></h3>
@using (Ajax.BeginForm("Form", "Home", new AjaxOptions
{
    HttpMethod = "POST",
    UpdateTargetId = "id1",
    LoadingElementId = "LoadingImage",
    OnSuccess = "onSuccess_Message",
    OnFailure = "onFailure_Message"
}))
{
    <table>
        <tr>
            <td>Enter ID: </td>
            <td>@Html.TextBoxFor(m => m.Id)</td>
        </tr>
        <tr>
            <td>Enter Name: </td>
            <td>@Html.TextBoxFor(m => m.Name)</td>
        </tr>
        <tr>
            <td>Addon: </td>
            <td>@Html.CheckBoxFor(m => m.Addon)</td>
        </tr>
        <tr>
            <td colspan="2"><input type="submit" value="Submit Form" /></td>
        </tr>
    </table>
    <div id="LoadingImage" style="display:none">Loading...</div>
    <div id="onSuccess_Message"></div>
    <div id="onFailure_Message"></div>
}
```

**** Execute the code and review the results**

Correct Data Entry:

☐ [Application name](#)

ID: 1
Name: Merve
Addon: False

Forms - Strongly Typed AJAX (Asynchronous)

Enter ID:

Enter Name:

Addon: ☐

© 2022 - My ASP.NET Application

Incorrect Data Entry:

☐ [Application name](#)

☐ [Application name](#)

Forms - Strongly Typed AJAX (Asynchronous)

Enter ID:

Enter Name:

Addon: ☒

© 2022 - My ASP.NET Application

Forms - Strongly Typed AJAX (Asynchronous)

Enter ID:

Enter Name:

Addon: ☒

© 2022 - My ASP.NET Application

PROCEDURE 3 – Pure Html Forms with Ajax and JQuery

In this method, you can not only send data from input controls but can also use html elements like `<p>...</p>`, `<span>...</span>` to send data to controllers. This is pure JQuery and AJAX query.

Step 1 – Create **Index2.cshtml** and create form like this.

```
<h3><b>Forms - Pure HTML and JQUERY</b></h3>

<table>
  <tr>
    <td>Enter ID: </td>
    <td><input type="text" id="Id" /></td>
  </tr>
  <tr>
    <td>Enter Name: </td>
    <td><input type="text" id="Name" /></td>
  </tr>
  <tr>
    <td>Addon: </td>
    <td><input type="checkbox" id="Addon" /></td>
  </tr>
  <tr>
    <td colspan="2"><button onclick="submit()">Submit Form</button></td>
  </tr>
</table>

<h4 style="color:purple" id="output"></h4>

<script src="~/Scripts/jquery-1.8.0.min.js" type="text/javascript"></script>
<script>
  function submit() {
    var data = {
      Id: $('#Id').val(),
      Name: $('#Name').val(),
      Addon: $('#Addon').is(':checked')
    };

    $.post("/Home/Form2", { sm: data }, function () {
      alert('Successfully Saved');
    });
  }
</script>
```

Step 2 – Go to HomeController and add following action method.

Let's create **Index2 ActionResult** as get method in Home Controller. This method should return Index.html view as default.

Next, let's create the **Form2 ActionResult** method. This method should create object from StudentModel class and use class items.

Thanks to jQuery, we can access the Form data and reprint the data within the same page.

Remember from the Javascript lesson! Script codes make changes by manipulating HTML tags.

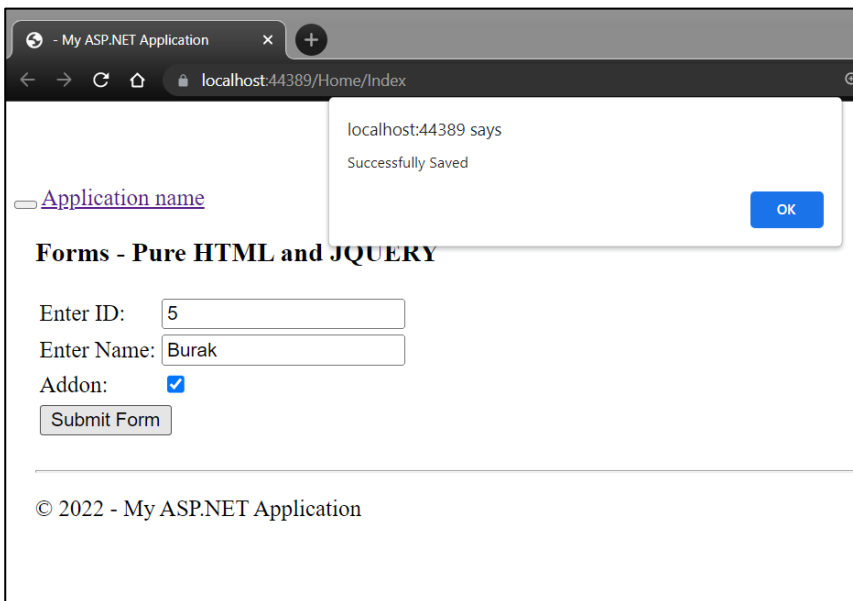
[HttpPost]

0 references

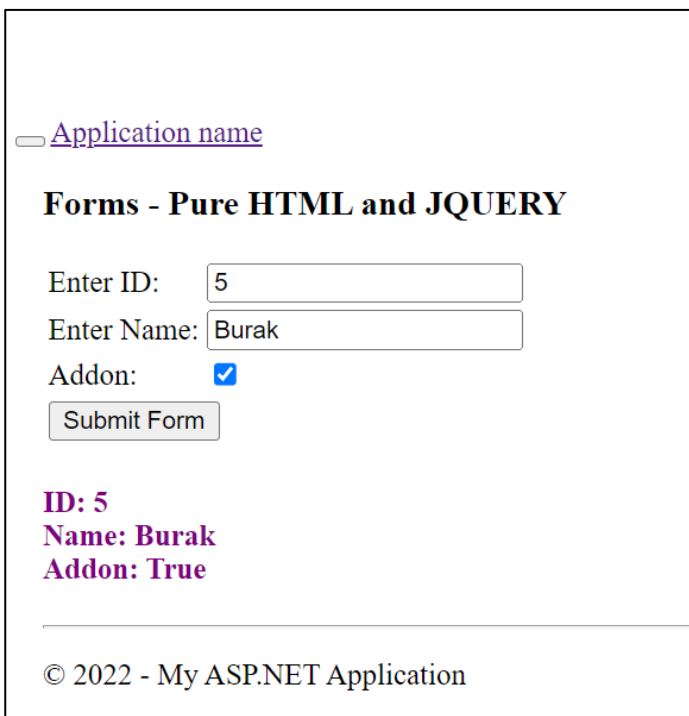
```
public ActionResult Form2(models.StudentModel sm)
{
    string value = "ID: " + Convert.ToString(sm.Id)
        + "<br />Name: " + sm.Name
        + "<br />Addon: " + Convert.ToString(sm.Addon);

    string s = $("'#output').html('" + value + "');";
    return JavaScript(s);
}
```

** Execute the code and review the results



The screenshot shows a web browser window with the address bar displaying "localhost:44389/Home/Index". A notification box in the top right corner states "localhost:44389 says Successfully Saved" with an "OK" button. The page content includes a link "Application name", a heading "Forms - Pure HTML and JQUERY", and a form with the following fields: "Enter ID:" with the value "5", "Enter Name:" with the value "Burak", and "Addon:" with a checked checkbox. A "Submit Form" button is located below the form. At the bottom of the page, the copyright notice "© 2022 - My ASP.NET Application" is visible.



This screenshot shows the rendered output of the form submission. It includes the same link "Application name" and heading "Forms - Pure HTML and JQUERY". Below the form fields, the submitted data is displayed in purple text: "ID: 5", "Name: Burak", and "Addon: True". The "Submit Form" button is still present. The copyright notice "© 2022 - My ASP.NET Application" remains at the bottom.

PROCEDURE 4 – Design a search page using Ajax and JQuery.

Let's design a search page using Ajax and JQuery. This page should be created to search for previously embedded Student information with the Name property.

Step 1 - Using the Student.cs class, we create a list with student information. For this, let's create the "JsonResult Students" method in HomeController. This method must create and use objects from the Student.cs class.

Since we will be searching based on name information, the object name is used as name.

HomeController/Students

```
[HttpPost]
public JsonResult Students(string name)
{
    var std_List = new List<StudentModel>
    {
        new StudentModel { Id = 1, Name = "Ayse", Addon = false },
        new StudentModel { Id = 2, Name = "Ali", Addon = true },
        new StudentModel { Id = 3, Name = "Can", Addon = false },
    };

    List<StudentModel> std_search = std_List.Where(x => x.Name == name).ToList();

    return Json(std_search, JsonRequestBehavior.AllowGet);
}
```

Step 2 - To design the web page to be searched, first the Search ActionResult method is created, then the View file named Search.cshtml is created under the Home folder. This view uses two homecontroller methods; JsonResult Students() and ActionResult Search().

HomeController/Search

Search ActionResult Method:

```
public ActionResult Search()
{
    return View();
}
```

Search.cshtml :

```
<script src="https://code.jquery.com/jquery-3.6.4.min.js"></script>

<input type="text" id="txtname" />
<input type="button" value="Search" id="list" />
<div id="Results"></div>

<script type="text/javascript">
    $(document).ready(function () {
        $('#list').click(function () {
            var name = $("#txtname").val();
            $.ajax({
                type: "post",
                url: "/Home/Students/",
                dataType: "json",
                data: { 'name': name },
                success: function (data) {
                    $("#Results").empty();
                    if (data.length > 0) {
                        $.each(data, function (index, item) {
                            $("#Results").append('<p>ID: ' + item.Id
                                + ', Name: ' + item.Name
                                + ', Addon: ' + item.Addon + '</p>');
                        });
                    } else {
                        $("#Results").append('<p>No results found.</p>');
                    }
                },
                error: function (error) {
                    console.log('Error:', error);
                }
            });
        });
    });
</script>
```

The search process allows you to make a search using pre-registered student information according to the features of the Student.cs class and display it on the same web page.

**** Execute the code and review the results**

ID: 2, Name: Ali, Addon: true

© 2023 - My ASP.NET Application

HTML Form

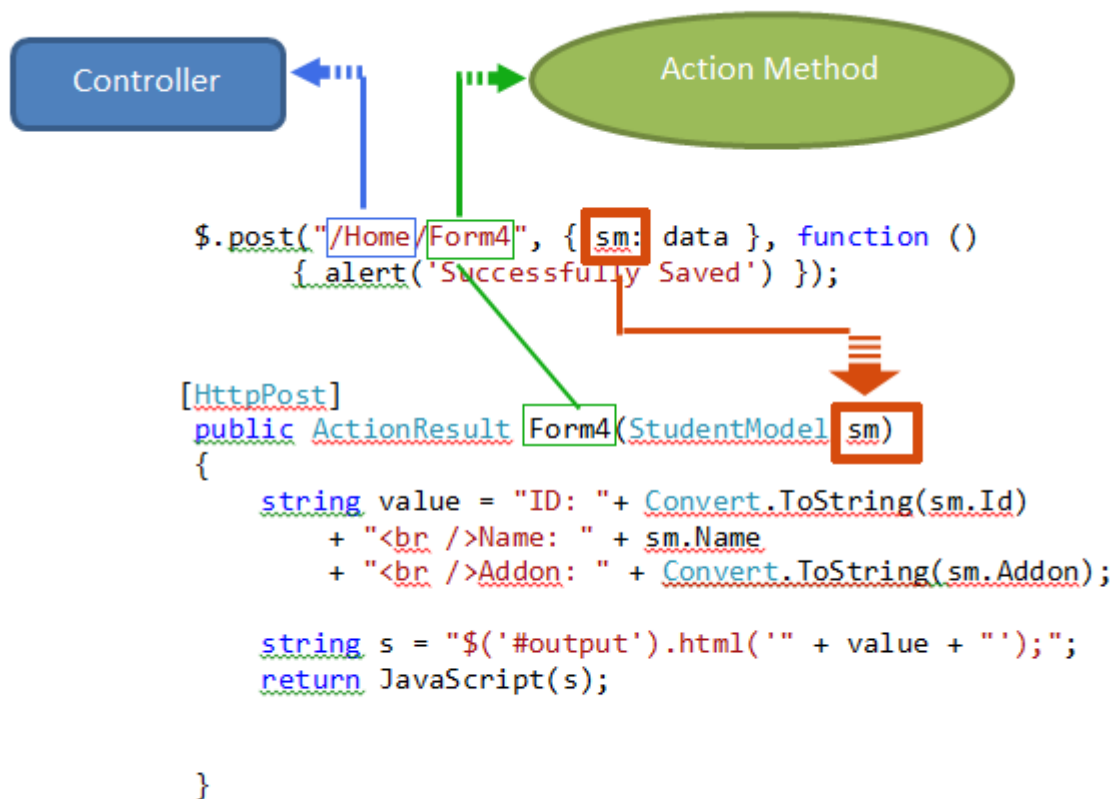
```
<td><input type="text" id="Id" /> </td>  
<td><input type="text" id="Name" /> </td>  
<td><input type="checkbox" id="Addon" /> </td>
```

jQuery

```
<script>  
function submit(){  
    var data = {  
        Id: $('#Id').val(),  
        Name: $('#Name').val(),  
        Addon: $('#Addon').is(':checked')  
    };  
    $.post("/Home/Form4", { sm: data }, function ()  
    { alert('Successfully Saved') });  
}  
</script>
```

Model Class

```
public class StudentModel  
{  
    public int Id { get; set; }  
    public string Name { get; set; }  
    public bool Addon { get; set; }  
}
```



T.A. MERVE GÜN